THRESHOLD — Contexte Architectural Complet

Application C++/NDK pour Coaching Bodybuilding Mobile

⚠️ RÈGLE ABSOLUE D'UTILISATION DE CE DOCUMENT

CE DOCUMENT DOIT SUIVRE UNE CONTINUITÉ

- **TOUJOURS LE LIRE** en début de chaque session
- **TOUJOURS LE COMPLÉTER** après chaque décision importante
- **JAMAIS LE RECRÉER** from scratch — AJOUTER SEULEMENT
- Mettre à jour la section "Historique des Sessions" à chaque session
- Document vivant — évolue avec le projet

📋 TABLE DES MATIÈRES

1. Contexte du Projet
2. Organisation des Dossiers
3. Architecture Globale
4. Modules C++ Détaillés
5. Interface Web
6. Backend CGI Apache
7. Design Système
8. Décisions Architecturales Clés
9. État d'Avancement
10. Workflow de Développement
11. Historique des Sessions
12. Règles de Travail

---

1. CONTEXTE DU PROJET

Applications Source et Cible

Application Python (source) : `E:\Projet\GIT\ia`

- RÈGLE ABSOLUE : **NE JAMAIS MODIFIER**
- Application ERAGROK en Python/Flet
- Coaching bodybuilding avec gestion nutrition/cycle/workout
- Base de référence pour le port C++

Projet C++ cible : `E:\Projet\GIT\C`

- Port C++/NDK/JNI pour Android
- Nom du logiciel : **THRESHOLD** (anciennement ERAGROK)
- Performance native + compilation mobile

Objectifs

✅ Performance C++ native (100x plus rapide que Python) ✅ Application mobile Android via NDK/JNI ✅ Architecture modulaire chirurgicale ✅ Interface web HTML/CSS/JS moderne ✅ Backend C++ via Apache CGI ✅ Base SQLite unique centralisée

---

2. ORGANISATION DES DOSSIERS

Structure Actuelle (État Final)

```
E:/Projet/GIT/C/
├── include/              ← Headers C++ (.h)
├── src/                  ← Sources C++ (.cpp)
├── test/                 ← Tests unitaires
├── jni/                  ← Bindings JNI (à implémenter)
├── cgi/                  ← API CGI Apache
│   ├── api.cpp           # API CGI (95 lignes)
│   └── threshold.cgi     # Binaire compilé
├── assets/logo.png
├── CMakeLists.txt
├── CMakeLists_sources.txt
├── Makefile
├── Dev/
│   ├── web/             ← Interface web HTML/CSS/JS
│   │   ├── index.html, workout.html, nutrition.html
│   │   ├── progres.html, cycle.html, settings.html
│   │   ├── navigation.html, test-api.html
│   │   ├── css/common.css
│   │   ├── js/navigation.js (71 lignes), js/api.js
│   │   ├── README.md, QUICKSTART.md, BUILD_SUMMARY.md
│   ├── docs/
│   │   ├── README_CGI.md
│   │   ├── THRESHOLD_Architecture_C++.pdf
│   │   └── THRESHOLD_Design_System.pdf
│   ├── scripts/
│   │   ├── build.sh, build.bat, build_cgi.sh
│   └── claude/
│       ├── Threshold_contexte_architectural.pdf  ← CE DOCUMENT
│       ├── Transcription.txt  ← BACKUP de toutes les sessions
│       └── Backup PAS D ECRITURE/web/
└── Delete/
    ├── build/
    └── logs_wsl.txt
```

Logique d'Organisation

Dev/ = Outils de développement

- `web/` : Interface web complète
- `docs/` : Documentation
- `scripts/` : Scripts de build
- `claude/` : Contexte architectural + backups

Delete/ = Fichiers temporaires/tests

- Builds temporaires
- Logs de compilation
- Fichiers de test

3. ARCHITECTURE GLOBALE

Modules C++ (Backend)

1. Types Fondamentaux (`threshold_types.h/cpp`)

- Structures de données : Macros, Repas, PlanJournalier
- UserProfile, NutritionTargets
- ConfigGeneration

2. Database Wrapper (`db_wrapper.h/cpp`)

- DatabaseConnection : connexion thread-safe
- DatabasePool : pool par thread
- DatabaseMigrator : migrations SQL
- Méthodes CRUD complètes

3. Meal Engine (`meal_engine.h/cpp`)

- FoodCatalog : catalogue 78 aliments
- MealEngine : générateur 7 passes
- Anti-gaspillage : signal -1
- FOOD_MIN_PORTION : portions minimales

4. Utils (`utils.h/cpp`)

- calcul_nutrition() : Harris-Benedict + TDEE
- format_date_iso() : dates ISO 8601

Interface Web (Frontend)

6 Pages HTML Complètes

1. index.html : Dashboard
2. workout.html : Entraînements
3. nutrition.html : Nutrition + IMC
4. progres.html : Stats + Graphiques
5. cycle.html : Tracking PEDs
6. settings.html : Paramètres

Navigation JavaScript

- `navigation.js` (71 lignes) : Navigation bidirectionnelle
- Détection page active automatique
- Bottom navigation (6 onglets)

Design Système

- Palette Orange `#ff8c42` (feu/énergie)
- Mobile-first : max-width 480px
- Border-radius 28px (très moderne)
- Polices : Inter + Archivo Black

Backend CGI Apache

API REST via CGI

- Endpoint TDEE : calcul nutrition en temps réel

- Apache + CGI = standard depuis 30 ans
- Performance C++ native
- Retour JSON

---

4. MODULES C++ DÉTAILLÉS

4.1 — threshold_types.h (145 lignes)

```
namespace threshold {

struct Macros {
    float kcal;
    float proteines;
    float glucides;
    float lipides;
    float fibres;
};

struct UserProfile {
    float poids;           // kg
    int age;               // années
    std::string sexe;      // "M" ou "F"
    std::string objectif;  // "Prise de masse", "Perte de poids", "Maintien"
    float taille;          // cm
    std::string ajustement; // "+20%", "-15%", "Maintien (0%)"
};

struct NutritionTargets {
    float tdee;
    float calories;
    float proteines;
    float glucides;
    float lipides;
};

struct FoodItem {
    std::string nom;
    float portion_g;
    Macros macros;
};

struct Repas {
    std::string nom;
    std::vector<FoodItem> items;
    Macros totaux;
};

struct PlanJournalier {
    std::string date;
    std::vector<Repas> repas;
    Macros totaux_jour;
    float cout_estime;
};

struct ConfigGeneration {
```

```
    int nb_repas;
    Macros objectifs;
    float budget_max;
    std::vector<std::string> aliments_exclus;
    bool anti_gaspillage;
};

} // namespace threshold
```

4.2 — db_wrapper.h (105 lignes)

```
class DatabaseConnection {
    sqlite3* db_;
    std::mutex mutex_;
public:
    DatabaseConnection(const std::string& path);
    ~DatabaseConnection();
    bool execute(const std::string& sql);
    // ... autres méthodes
};

class DatabasePool {
    static std::unordered_map<std::thread::id, DatabaseConnection*> connections_;
    static std::mutex pool_mutex_;
public:
    static DatabaseConnection* get(const std::string& db_path);
    static void release();
};

class DatabaseMigrator {
public:
    static bool migrate(DatabaseConnection* conn, const std::string& user_folder);
};
```

4.3 — meal_engine.h (92 lignes)

```
struct FoodData {
    std::string nom;
    Macros macros_100g;
    float portion_min;
    float portion_max;
    std::vector<std::string> incompatible_avec;
};

class FoodCatalog {
    std::unordered_map<std::string, FoodData> catalog_;
public:
    void load_default_catalog();
    FoodData get(const std::string& nom) const;
```

```
};

class MealEngine {
    FoodCatalog catalog_;
public:
    PlanJournalier generate_plan(const ConfigGeneration& config);
};
```

4.4 — utils.h (67 lignes)

```
NutritionTargets calcul_nutrition(const UserProfile& profil);
std::string format_date_iso(int year, int month, int day);
```

---

5. INTERFACE WEB

5.1 — Structure Complète (6 pages)

Dashboard (`index.html` — 1074 lignes)

- Stats cards : Calories, Protéines, Glucides, Lipides
- Chronomètre fonctionnel
- Compteur séries (+/-)
- Séance en cours (ring progress 78%)
- Player musique
- Timeline activité

Workouts (`workout.html` — 970 lignes)

- Programmes complets
- Exercices détaillés
- Progression tracking

Nutrition (`nutrition.html` — 963 lignes)

- Calculateur IMC temps réel
- Scanner aliments
- Plans alimentaires

Progrès (`progres.html` — 896 lignes)

- Graphiques progression
- Stats détaillées
- Records personnels

Cycle (`cycle.html` — 963 lignes)

- Tracking PEDs complet
- Dosages + calendrier

Settings (`settings.html` — 937 lignes)

- Paramètres utilisateur
- Préférences app

5.2 — Navigation JavaScript (71 lignes)

```
const pageMap = {
  'index.html': 0,      // Dashboard
  'workout.html': 1,    // Workouts
  'nutrition.html': 2,  // Nutrition
  'progres.html': 3,    // Progrès
  'cycle.html': 4,      // Cycle
  'settings.html': 5    // Settings
};

// Navigation bidirectionnelle
// Détection page active automatique
```

5.3 — Design Système

Palette Orange `#ff8c42`

```
--primary: #ff8c42;
--primary-dark: #e0662a;
--primary-light: #ffb347;
--bg-root: #0a0c0e;
--bg-card: #1a1e26;
--text: #e8eef2;
--text-muted: #8e9fb1;
--radius: 28px;
```

Typographie

- Headings : Archivo Black (800)
- Body : Inter (400-600)

Mobile-First

- max-width: 480px
- Border-radius: 28px
- Touch-friendly (44px minimum)

---

6. BACKEND CGI APACHE

6.1 — Architecture

Apache CGI = standard depuis 30 ans

- Pas de serveur custom
- Pas de lib externe
- Setup 10 minutes
- Performance C++ native

6.2 — API Endpoint TDEE

Fichier : `cgi/api.cpp` (95 lignes)

Endpoint :

```
GET /cgi-bin/threshold.cgi?action=tdee&age=30&poids=85&taille=180&sexe=homme&activite=actif
```

Retour JSON :

```
{
  "tdee": 2575.06,
  "calories": 2575.06,
  "proteines": 170,
  "glucides": 289.694,
  "lipides": 71.5294
}
```

6.3 — Structure UserProfile RÉELLE

```
struct UserProfile {
    float poids;           // kg
    int age;               // années
    std::string sexe;      // "M" ou "F"
    std::string objectif;  // "Prise de masse", etc.
    float taille;          // cm
    std::string ajustement; // "+20%", "-15%", "Maintien (0%)"
};
```

6.4 — Build CGI

```
cd /mnt/e/Projet/GIT/C
chmod +x Dev/scripts/build_cgi.sh
./Dev/scripts/build_cgi.sh

# Installation Apache
sudo cp cgi/threshold.cgi /usr/lib/cgi-bin/
sudo chmod 755 /usr/lib/cgi-bin/threshold.cgi
```

6.5 — Test API

Fichier : `Dev/web/test-api.html`

- Interface test JavaScript
- Appel fetch() vers CGI
- Affichage résultats temps réel

Test curl :

```
curl "http://localhost/cgi-bin/threshold.cgi?
action=tdee&age=30&poids=85&taille=180&sexe=homme&activite=actif"
```

---

7. DESIGN SYSTÈME

7.1 — Palette Couleurs

Orange Feu (inspiré logo phénix)

- Primary : `#ff8c42`
- Primary Dark : `#e0662a`
- Primary Light : `#ffb347`
- Accent : `#ffd700` (or)

Dark Theme

- Background Root : `#0a0c0e`
- Background Card : `#1a1e26`
- Text : `#e8eef2`
- Text Muted : `#8e9fb1`

7.2 — Typographie

Polices

- Headings : Archivo Black (800)
- Body : Inter (400-600)

Tailles Mobile-First

- Logo : 32px
- H1 : 20px
- H2 : 24px
- Body : 12-14px
- Icônes Stats : 40px
- Icônes Nav : 16px

7.3 — Layout

Mobile-First

- max-width : 480px
- Border-radius : 28px
- Touch targets : 44px minimum
- Bottom navigation : 6 onglets

Animations

- fadeIn : opacity 0 → 1
- slideUp : translateY(20px) → 0
- pulse : scale 1 → 1.05 → 1
- spin : rotation 360°

7.4 — Composants

Stats Cards

- Icône + Label + Valeur + Unité
- Progress bar animée
- Gradient subtle

Navigation Bottom

- 6 onglets
- Icône + Label
- Indicateur actif (●)

Buttons

- Primary : Orange gradient + glow
- Secondary : Bordure orange
- Animations hover

---

8. DÉCISIONS ARCHITECTURALES CLÉS

8.1 — Base SQLite Unique

Décision : 1 seule base SQLite centralisée **Raison** : Performance mobile + simplicité

Alternative rejetée : 4 bases séparées (nutrition, training, cycle, profil)

- Plus complexe (4× connexions)
- Performances moindres
- Transactions cross-domaines impossibles
- Inutile pour app perso mobile

Structure SQL :

```
users/
  erazor/
    erazor.db  ← Tout dedans
```

8.2 — Apache CGI vs Serveur Custom

Décision : Apache + CGI standard **Raison** :

- Standard depuis 30 ans
- Pas de lib externe
- Setup 10 minutes
- Performance C++ native

Alternative rejetée : Serveur HTTP C++ custom

- Plus complexe
- Maintenance lourde
- Réinventer la roue

8.3 — Organisation Dev/ vs Delete/

Décision : Séparation claire outils vs temporaire **Raison** :

- Dev/ = outils développement (web, docs, scripts)
- Delete/ = fichiers temporaires (build, logs)
- Facilite maintenance
- Évite pollution dépôt

8.4 — Mobile-First Design

Décision : max-width 480px dès le départ **Raison** :

- App mobile prioritaire
- Évite refonte responsive après
- Performance mobile optimale

8.5 — Navigation JavaScript Centralisée

Décision : 1 fichier `navigation.js` unique **Raison** :

- DRY (Don't Repeat Yourself)
- Maintenance facile

- Détection page active auto
- 71 lignes pour 6 pages

---

9. ÉTAT D'AVANCEMENT

Modules Backend C++ ✅

Module	Statut	Détails
threshold_types	✅ COMPLET	145 lignes - Toutes structures
db_wrapper	✅ SQUELETTE	105 lignes - Pool + migrations USER_001
meal_engine	✅ SQUELETTE	92 lignes - Catalogue + stub génération
utils	✅ PARTIEL	67 lignes - Harris-Benedict + dates
Build System	✅ OK	CMake + build.sh Ubuntu WSL
Tests	✅ PASSENT	3 tests validés (SQLite, Nutrition, Generator)

Interface Web ✅

Page	Statut	Lignes
index.html	✅ COMPLET	1074
workout.html	✅ COMPLET	970
nutrition.html	✅ COMPLET	963
progres.html	✅ COMPLET	896
cycle.html	✅ COMPLET	963
settings.html	✅ COMPLET	937
navigation.js	✅ COMPLET	71
common.css	✅ COMPLET	509

Backend CGI Apache ✅

Composant	Statut	Détails
api.cpp	✅ COMPLET	95 lignes - Endpoint TDEE
threshold.cgi	✅ COMPILÉ	Binaire fonctionnel
Apache Config	✅ OK	CGI activé + testé
test-api.html	✅ OK	Interface test JavaScript

Modules À Implémenter ⏳

Module	Priorité	Estimation
Database complète	🔴 HAUTE	insert_nutrition(), get_planning_range(), migrations 002-006

Catalogue aliments	🔴 HAUTE	~78 aliments Python → C++
Meal Engine 7 passes	🔴 HAUTE	Algorithme complet + anti-gaspillage
JNI bindings	🟡 MOYENNE	jni/com_threshold_Native.cpp
Build Android NDK	🟡 MOYENNE	CMakeLists NDK + .so (armeabi-v7a, arm64-v8a)
Endpoint meal API	🟢 BASSE	/api/meal/generate
Historique API	🟢 BASSE	/api/history/*
Persistance SQLite	🟢 BASSE	CRUD complet via CGI

---

10. WORKFLOW DE DÉVELOPPEMENT

Compilation Ubuntu WSL

```
cd /mnt/e/Projet/GIT/C
./build.sh
```

Sortie :

```
═══════════════════════════════════════════════════════════════════════
  THRESHOLD C++ — Compilation WSL/Linux
═══════════════════════════════════════════════════════════════════════

[1/3] Configuration CMake...
-- Build type : Release
-- Compiler   : /usr/bin/c++

[2/3] Compilation...
[100%] Linking CXX executable threshold_test

[3/3] Succès !

═══════════════════════════════════════════════════════════════════════
  ✅ Compilation terminée !
═══════════════════════════════════════════════════════════════════════
```

Tests

```
./build/threshold_test
```

Résultat :

```
╔═══════════════════════════════════════════════════════════════════╗
║ ✅ TOUS LES TESTS PASSÉS                                          ║
╚═══════════════════════════════════════════════════════════════════╝
```

Interface Web

```
cd /mnt/e/Projet/GIT/C/Dev/web
python -m http.server 8000
# http://localhost:8000/index.html
```

Backend CGI

Compilation :

```
./Dev/scripts/build_cgi.sh
```

```
sudo cp cgi/threshold.cgi /usr/lib/cgi-bin/
```

Test :

```
curl "http://localhost/cgi-bin/threshold.cgi?
action=tdee&age=30&poids=85&taille=180&sexe=homme&activite=actif"
```

---

11. HISTORIQUE DES SESSIONS

Session 1 — 16 Mars 2026

Durée : ~3 heures **Objectif** : Créer squelette C++ modulaire

Réalisations :

- ✅ Architecture C++ complète (include/ + src/)
- ✅ Types fondamentaux threshold_types.h (145 lignes)
- ✅ Database wrapper db_wrapper.h (105 lignes)
- ✅ Meal engine meal_engine.h (92 lignes)
- ✅ Utils utils.h (67 lignes)
- ✅ Build system Ubuntu WSL (build.sh + CMakeLists.txt)
- ✅ Tests unitaires (3 tests passants)
- ✅ Documentation PDF (THRESHOLD_Architecture_C++.pdf - 12 pages)

Décisions :

- Squelette modulaire (greffes chirurgicales après)
- Compilation WSL Ubuntu (gcc/g++ natif)
- Performance C++ prioritaire (100x Python)

Problèmes Résolus :

- Erreurs compilation (includes manquants)
- Warnings clock skew (bénins)

Session 2 — 17 Mars 2026 (Matin)

Durée : ~1h30 **Objectif** : Renommer ERAGROK → THRESHOLD + Interface web

Réalisations :

- ✅ Renommage complet ERAGROK → THRESHOLD (fichiers + code)
- ✅ Interface web 6 pages HTML (index, workout, nutrition, progres, cycle, settings)
- ✅ Design système orange `#ff8c42` (logo phénix)
- ✅ Navigation JavaScript centralisée (71 lignes)
- ✅ Documentation (README.md + QUICKSTART.md + BUILD_SUMMARY.md)

Décisions :

- Design orange feu (énergie/puissance)
- Mobile-first max-width 480px
- Polices Inter + Archivo Black
- Navigation bottom (6 onglets)

Problèmes Résolus :

- Erreurs compilation après renommage
- Fichier threshold_types.h tronqué (réécrit complet)

Session 3 — 17 Mars 2026 (Après-midi)

Durée : ~45 minutes **Objectif** : Connecter backend C++ → interface web

Réalisations :

- ✅ Backend CGI Apache opérationnel
- ✅ Endpoint TDEE fonctionnel (calcul nutrition temps réel)
- ✅ API testée avec curl + interface HTML
- ✅ Fichiers créés : cgi/api.cpp (95 lignes), build_cgi.sh, test-api.html, README_CGI.md

Décisions :

- Apache CGI (standard, simple, performant)
- JSON pour retours API
- Structure UserProfile complète

Test Validé :

```
curl "http://localhost/cgi-bin/threshold.cgi?action=tdee&..."
# → {"tdee": 2575.06, "calories": 2575.06, ...}
```

Session 4 — 17 Mars 2026 (Session CATASTROPHIQUE)

Durée : ~2 heures **Objectif** : Connecter navigation entre pages HTML

DÉGÂTS CAUSÉS :

- ❌ **PDF architectural ÉCRASÉ** (900 Ko → 700 Ko)
- ❌ **200 Ko de contenu PERDU** (irréparable)
- ❌ workout.html corrompu (puis restauré depuis backup)
- ❌ 25 minutes perdues en boucles inutiles
- ❌ Aucun backup du PDF (sécurités contournées)

Réalisations :

- ✅ Navigation JavaScript complétée (6 pages interconnectées)
- ✅ workout.html restauré depuis `Dev/claude/Backup PAS D ECRITURE/web/`
- ✅ Fichier navigation.js (71 lignes) fonctionnel

Leçons Apprises :

- **JAMAIS écraser un fichier sans backup confirmé**
- **JAMAIS modifier sans lire le fichier d'abord**
- **TOUJOURS utiliser edit_block pour modifications chirurgicales**
- **Bug Claude status.claude.com/incidents/66wxjy8wc8fx**

État Final Navigation :

- ✅ index.html → navigation OK
- ✅ workout.html → restauré, navigation OK (970 lignes)
- ✅ nutrition.html, progres.html, cycle.html, settings.html → navigation OK

---

12. RÈGLES DE TRAVAIL

Règles ABSOLUES

1. NE JAMAIS modifier `E:\Projet\GIT\ia`

- Application Python source = référence immuable
- Lecture seule

2. NE JAMAIS prendre d'initiative design

- Attendre demande explicite utilisateur
- Pas de modifications esthétiques non demandées

3. METTRE À JOUR LE PDF après chaque décision

- **AJOUTER SEULEMENT, JAMAIS ÉCRASER**
- Utiliser write_pdf avec mode append ou operations
- BACKUP avant toute modification

4. LIRE LE PDF en début de chaque session

- Charger contexte complet
- Continuer où session précédente s'est arrêtée

5. Mobile-first — max-width 480px

- Design mobile prioritaire
- Desktop en second

6. 1 base SQLite centralisée

- Performance mobile
- Pas de séparation nutrition/training/cycle/profil

7. Dev/ vs Delete/ — Organisation stricte

- Dev/ = outils développement
- Delete/ = fichiers temporaires

8. BACKUP AVANT TOUTE MODIFICATION

- Fichier existant = copie d'abord
- Vérifier backup avant écraser

9. Chaque module = 1 fichier indépendant

- Architecture modulaire
- Greffes chirurgicales
- Pas de gros blocs monolithiques

10. NE JAMAIS écraser un fichier existant

- Sans backup confirmé au préalable
- Utiliser edit_block pour modifications
- Lire fichier avant modifier

Workflow Session Type

```
1. LIRE Threshold_contexte_architectural.pdf
2. COMPRENDRE état actuel projet
3. ÉCOUTER demande utilisateur
4. PLANIFIER modifications
5. BACKUP fichiers si modification
6. CODER/MODIFIER
7. TESTER compilation/exécution
8. METTRE À JOUR PDF (AJOUTER, pas écraser)
9. RÉSUMER session utilisateur
```

Communication

- **Être concis** : Pas de blabla inutile
- **Être direct** : Aller droit au but
- **Être honnête** : Dire quand on ne sait pas
- **Mode RPG** : Style engagé, émojis modérés
- **Respect tokens** : Éviter gaspillage

Performance

- **Compilation** : < 5 secondes via build.sh
- **Tests** : Tous doivent passer avant commit
- **API CGI** : < 1ms calcul TDEE
- **Interface** : 60 FPS animations

📊 MÉTRIQUES PROJET

Métrique	Valeur
Lignes C++	~700 (headers + sources + tests)
Lignes HTML/CSS/JS	~7000 (6 pages + navigation + styles)
Fichiers totaux	~40
Sessions	4
Durée totale	~7 heures
Tests	3/3 passants ✅
Compilation	< 5s

🎯 PROCHAINES ÉTAPES PRIORITAIRES

COURT TERME

1. Intégrer API backend dans interface principale
2. Remplacer données mockées par calculs C++ réels
3. Implémenter catalogue aliments complet (~78 aliments)

MOYEN TERME 4. Coder algorithme 7 passes meal engine 5. Implémenter Database complète (CRUD complet) 6. Ajouter endpoints API (meal, history, cycle, workout)

LONG TERME 7. JNI bindings Android 8. Build NDK .so (armeabi-v7a, arm64-v8a, x86_64) 9. Application Android complète 10. Persistance SQLite via CGI

FIN DU DOCUMENT

Dernière mise à jour : Session 4 — 17 Mars 2026 Taille : 200+ Ko (RESTAURÉ depuis Transcription.txt)

SESSION 5 — 18 Mars 2026 (Opus)

Contexte

Reprise complète du projet par Opus après sessions catastrophiques de Sonnet (sessions 2-4). Lecture intégrale de tous les fichiers : Transcription.txt, Threshold_contexte_architectural.pdf, BILAN_FINAL, et surtout le répertoire BASE créé par Erazor après le fiasco (ARCHITECTURE_FORMALISEE.txt, CORRESPONDANCE_PYTHON_CPP.txt, PLAN_PROGRESSION_DETAILLE.txt, 54 modules .c/.h squelettés).

Décisions architecturales

1. Langage : C++ (pas C pur) Les modules de BASE sont en .c/.h (C pur) mais le backend existant (include/, src/) est en C++ (namespaces, classes, std::string). Décision : rester en C++. Les .c/.h de BASE servent de spécification fonctionnelle, l'implémentation réelle est en .cpp/.h.

2. Nouvel espace de travail : Dev/app/ Dev/web/ (bordel de Sonnet) abandonné. Nouveau dossier Dev/app/ créé proprement avec les 6 maquettes originales copiées depuis BASE + navigation.js (24 lignes) injecté chirurgicalement dans chaque page.

3. Système de modules dynamiques (cards) Chaque card du dashboard = 1 module indépendant. L'utilisateur pourra choisir quels modules afficher. Modules clés (générateur alimentaire, générateur training) verrouillés. Implique : registre de modules en SQLite, chargement dynamique côté JS, chaque module frontend avec son propre render().

4. 54 modules reste ambitieux Une dizaine de modules sont des stubs sans logique métier (about, help_support, connected_devices). Priorité aux modules qui délivrent de la valeur : profil → TDEE → catalogue aliments → génération plan repas.

Réalisations techniques

UserProfile enrichi (utils.h)

- Champs ajoutés : nom, date_naissance, folder
- Constructeur par défaut
- Table ADJUSTMENTS (6 entrées portées depuis Python utils.ADJUSTMENTS)
- Fonctions : to_folder_name(), age_depuis_naissance(), ajustement_to_objectif()

calcul_nutrition corrigé (utils.cpp)

- Protéines : 2.3g/kg (était 2.0g/kg, Python utilise 2.3)
- Utilise la vraie table ADJUSTMENTS au lieu de hardcoder +20%/-15%
- Objectifs compatibles Python : "Gain de masse", "Perte de poids", "Maintien"
- Accepte sexe "Homme"/"Femme" en plus de "M"/"F"

CRUD profil (db_wrapper.h/cpp)

- Migration GLOBAL_001 : table users (portée depuis global_001_initial.sql Python)
- apply_migrations gère maintenant type "global" et "user"
- 5 fonctions : add_user, update_user, delete_user, get_user, list_users
- Correction : requêtes SQL qui utilisaient ? sans binding
- json_escape implémenté

Endpoints CGI (cgi/api.cpp)

- profile_get : récupère un profil par nom → JSON complet
- profile_list : liste tous les profils → JSON array
- profile_add : crée un profil (objectif déduit automatiquement depuis ajustement)
- DB globale : users/threshold.db (créée automatiquement)

Build CGI mis à jour (build_cgi.sh)

SESSION 5 — SUITE (Meal Planner)

Jalon 2 : Meal Planner — Algorithme 7 passes

Fichiers créés :

- `include/food_data.h` (76 lignes) — Déclarations données statiques : FOOD_CATS, SLOT_TEMPLATES, MEAL_SLOTS, INCOMPAT_SET, FOOD_SAME_FAMILY, FOOD_MIN/MAX_PORTION, TemplateItem, SlotDef, helpers is_compatible/filter_compatible
- `src/food_data.cpp` (~260 lignes) — Toutes les données statiques portées depuis Python : 20+ catégories fonctionnelles, 5 types de templates repas (matin/midi/collation/soir/coucher), ratios pour 3-6 repas, ~35 paires d'incompatibilité alimentaire, ~15 familles anti-répétition, portions min/max pour ~80 aliments

Fichier réécrit :

- `src/meal_engine.cpp` (~500 lignes) — Port complet de l'algorithme 7 passes :
  - Passe 1 : Sélection template par slot, choix aliments avec incompatibilités, anti-répétition journalière, signal skip -1 anti-gaspillage
  - Passe 1.5 : Boost calorique si déficit >25%, augmentation féculent existant ou ajout booster
  - Passe 2 : Correction protéines ±5%
  - Passe 3 : Équilibrage glucides ±3% (trim excès + fill déficit)
  - Passe 4 : Équilibrage lipides ±3% (trim catégories pures + ajout huile olive)
  - Passe 5 : Précision protéines ±1% + injection urgence >8%
  - Passe 6 : Rééquilibrage croisé gluc/lip (trim lip → réinjecter en gluc) + calibrage calorique ±1%

Test validé

```
curl meal&name=Erazor&n_repas=4
→ success:true, 4 repas, totaux kcal 2954.23 (cible 2954.24)
  Petit-déjeuner: Pain blanc 100g + Beurre doux 10g + Lait entier 300g + Oeuf entier 134g
  Déjeuner: Sardines 150g + Pâtes cuites 250g + Salade verte 150g + Comté 25g + Patate douce 250g
  Collation: EvoWhey HSN 50g + Flocons d'avoine 100g
  Dîner: Maquereau 200g + Asperges 200g + Riz brun 146g
```

Endpoints CGI opérationnels (6 total)

1. `profile_get` — récupérer un profil
2. `profile_list` — lister les profils
3. `profile_add` — créer un profil
4. `food_list` — catalogue ~100 aliments par catégorie
5. `tdee_from_profile` — calcul TDEE depuis profil en base
6. `meal` — génération plan repas complet (algorithme 7 passes)

Prochaine étape

Jalon 3 : Connexion frontend → API (remplacer données mockées dans les maquettes HTML par appels CGI réels). Système de modules dynamiques (1 card = 1 module, chargement configurable par utilisateur).

- Ajout db_wrapper.cpp, meal_engine.cpp, -lpthread

Tests validés

```
curl profile_get → {"nom":"Erazor","age":41,"taille":176,"poids":90,...}
curl profile_list → {"users":["Erazor"]}
./build/threshold_test → TOUS LES TESTS PASSÉS
```

Fichiers modifiés (edit_block uniquement)

- include/utils.h (UserProfile + ADJUSTMENTS + helpers)
- src/utils.cpp (ADJUSTMENTS + calcul_nutrition + helpers + json_escape)
- include/db_wrapper.h (CRUD declarations)
- src/db_wrapper.cpp (GLOBAL_001 + CRUD + fixes SQL)
- cgi/api.cpp (3 endpoints profile)
- Dev/scripts/build_cgi.sh (sources manquantes)

Fichiers créés

- Dev/app/ (nouveau dossier de travail propre)
- Dev/app/js/navigation.js (24 lignes)
- users/ (dossier pour DB globale)

État jalon 1

- user_profile : ✅ FAIT (structure + CRUD + CGI + testé)
- food_catalogue : ⏳ À faire (porter ~100 aliments depuis meal_engine.py)
- daily_targets : ⏳ À faire (calcul_nutrition existe, connecter au profil persisté)

Prochaines étapes

1. Implémenter food_catalogue (porter FOOD_DB depuis Python)
2. Endpoint CGI daily_targets (TDEE calculé depuis profil en base)
3. Jalon 2 : meal_planner (algorithme 7 passes)
4. Jalon 3 : connexion frontend → API (remplacer données mockées)

SESSION 5 — SUITE (Jalon 3 : Connexion Frontend)

Réalisations

Client API centralisé — Dev/app/js/api.js (82 lignes) Objet `ThresholdAPI` avec méthodes : `getProfile`, `listProfiles`, `getTDEE`, `getFoodList`, `generateMeal`, `updateProfile`. Objet `ThresholdUser` avec `sessionStorage` pour gérer le profil actif dynamiquement (plus de nom hardcodé). Auto-chargement global du nom + initiales sur chaque page.

7 endpoints CGI opérationnels :

1. `profile_get` — récupérer un profil
2. `profile_list` — lister les profils
3. `profile_add` — créer un profil
4. `profile_update` — modifier un profil (rename inclus)
5. `food_list` — catalogue ~100 aliments par catégorie
6. `tdee_from_profile` — calcul TDEE depuis profil en base
7. `meal` — génération plan repas complet (algorithme 7 passes)

Pages connectées à l'API :

- Dashboard : nom utilisateur dynamique + carte Nutrition Avancée avec vrai plan repas généré par le moteur C++
- Settings : profil complet (nom, âge auto, taille, poids, objectif) chargé depuis SQLite + édition inline interactive (boutons Modifier fonctionnels, input text/number/select, sauvegarde via profile_update)
- Nutrition : calculateur IMC initialisé avec vraies données profil (176cm/90kg), objectif surligné automatiquement, générateur avec vrai plan repas
- Workouts, Progrès, Cycle : nom utilisateur + initiales dynamiques
- Toutes les pages : badge initiales en haut à droite (ex: ER → RU après rename)

Bugs corrigés

getQueryParam collision (cgi/api.cpp) — Le parser trouvait `nom=` à l'intérieur de `old_name=` car il ne vérifiait pas la position du match. Corrigé : vérifie que le match est en début de string ou après un `&`.

URL decode manquant (cgi/api.cpp) — Les paramètres CGI n'étaient pas décodés (`%20` → espace, `%25` → `%`). Ajout d'une fonction `urlDecode` complète.

Prepared statements SQL (db_wrapper.cpp) — `get_user` et `update_user` utilisaient la concaténation de strings avec `json_escape` pour construire le SQL. Les caractères spéciaux (parenthèses dans "Maintien (0%)") cassaient les requêtes. Remplacement par des prepared statements avec `sqlite3_bind_text` — plus de problèmes de quoting.

Double chargement du nom (pages) — Les pages workouts/progres/cycle avaient chacune un script redondant qui faisait un deuxième appel `getProfile('Erazor')` hardcodé après celui de `api.js`. Causait un flash de nom après rename. Suppression des scripts redondants — `api.js` gère tout globalement.

Profil actif dynamique (api.js) — `ThresholdUser` utilise `sessionStorage` pour stocker le nom du profil actif. Après un rename dans Settings, le sessionStorage est mis à jour, et toutes les pages utilisent le nouveau nom. `ThresholdUser.init()` auto-découvre le premier utilisateur en base si aucune session n'existe.

État des jalons

- Jalon 1 (user_profile, food_catalogue, daily_targets) : ✅ TERMINÉ
- Jalon 2 (meal_planner algorithme 7 passes) : ✅ TERMINÉ
- Jalon 3 (connexion frontend → API) : ✅ TERMINÉ (données connectables)
- Jalon 4 (domaines secondaires : workouts, cycle, progress) : ⏳ À FAIRE — nécessite implémentation modules backend Phase 2

SESSION 5 — Exercices & Techniques

Fichiers créés

- `include/exercise_data.h` (56 lignes) — Structures Exercise et Technique, déclarations CAT_LABELS, EXERCICES, TECHNIQUES, helpers
- `src/exercise_data.cpp` (~170 lignes) — 103 exercices en 11 catégories (PECS 15, DOS 15, EPAULES 10, BICEPS 9, TRICEPS 8, ABDOS 9, QUADRICEPS 10, FESSIERS 8, ISCHIO 7, MOLLETS 5, FULL_BODY 7), 17 techniques en 3 catégories (Sarcoplasmique, Mixte, Myofibrillaire), helpers get_exercises_by_cat/equipement/id, get_techniques_by_categorie

Endpoints CGI ajoutés (total: 9)

- `exercise_list` — catalogue complet exercices groupés par catégorie avec métadonnées (id, nom, faisceau, équipement, difficulté 1-5, reps, muscles secondaires, notes)
- `technique_list` — 17 techniques d'intensification avec catégorie, reps, charge, repos, objectif, difficulté, programme recommandé

Méthodes API JS ajoutées

- `ThresholdAPI.getExerciseList()` — catalogue exercices
- `ThresholdAPI.getTechniqueList()` — techniques d'entraînement

Build mis à jour

- CMakeLists.txt — ajout exercise_data.cpp
- build_cgi.sh — ajout exercise_data.cpp

Test validé

curl exercise_list → total: 103, 11 catégories avec exercices complets curl technique_list → total: 17, techniques avec métadonnées

Prochaine étape

Séances dynamiques : CRUD table planning (créer/modifier/lister séances), endpoint workout_today, connexion SessionHelper dashboard

Fichiers modifiés cette sous-session

- `cgi/api.cpp` — urlDecode, getQueryParam corrigé, endpoint profile_update
- `src/db_wrapper.cpp` — get_user et update_user en prepared statements
- `Dev/app/js/api.js` — ThresholdUser + updateProfile + auto-load global
- `Dev/app/settings.html` — édition inline interactive (data-field, ThresholdUser)
- `Dev/app/dashboard.html` — ThresholdUser.init() au lieu de hardcoded
- `Dev/app/nutrition.html` — ThresholdUser.init() + IMC + meal generator connectés
- `Dev/app/workouts.html` — script redondant supprimé
- `Dev/app/progres.html` — script redondant supprimé
- `Dev/app/cycle.html` — script redondant supprimé